

Frequency Fitness Assignment as a Research Direction

Thomas Weise (汤卫思)

tweise@hfu.edu.cn · <https://thomasweise.github.io>

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Outline



1. Introduction
2. Research Field: Optimization
3. Research Direction: Frequency Fitness Assignment
4. Our Results
5. Future Works
6. Summary
7. Advertisement





Introduction



Introduction



- Our team has two general research directions: Optimization⁷⁷ and Artificial Intelligence (AI).

Introduction



- Our team has two general research directions: Optimization⁷⁷ and Artificial Intelligence (AI).
- My students and I are members of the Optimization Direction.

Introduction



- Our team has two general research directions: Optimization⁷⁷ and Artificial Intelligence (AI).
- My students and I are members of the Optimization Direction.
- Today, I therefore want to first give a short introduction into the field of optimization, before delving into one of our concrete research topics, namely Frequency Fitness Assignment (FFA).



Research Field: Optimization



What is Optimization?

- There are two ways to look at optimization.



What is Optimization?



- The economic view.

Optimization Problem

An optimization problem is a situation which requires deciding for one choice from a set of possible alternatives in order to reach a predefined or required benefit at minimal costs.

What is Optimization?



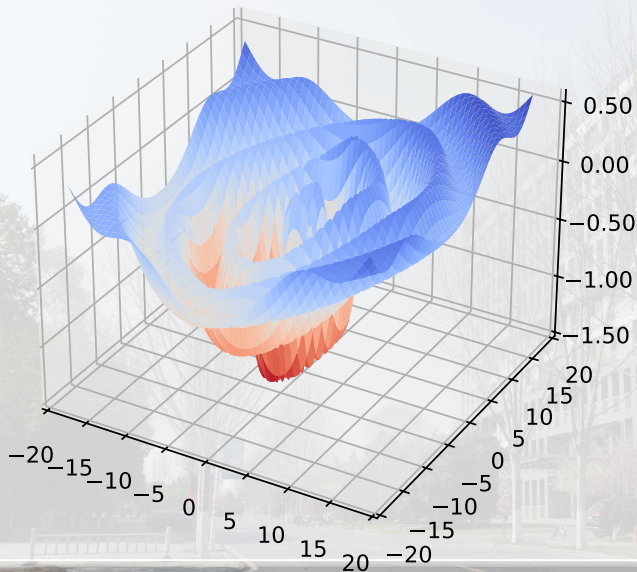
- The mathematical view.

Optimization Problem

An optimization problem is a situation which requires deciding for one choice from a set of possible alternatives in order to reach a predefined or required benefit at minimal costs.

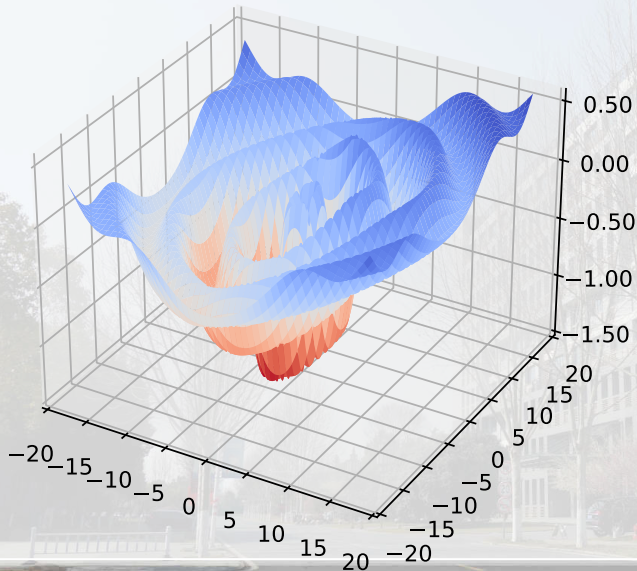
Solving an optimization problem requires finding an input element x^* within a set $\mathbb{X}$ of allowed elements for which a mathematical function $f: \mathbb{X} \mapsto \mathbb{R}$ takes on the smallest possible value.

Example: Function Optimization



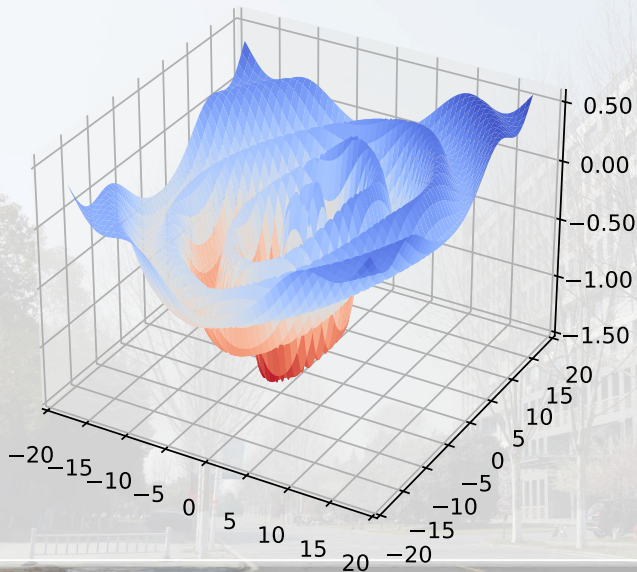
- In the field of continuous optimization, we could literally try to find the **minimum** of a mathematical function $f : \mathbb{R}^n \mapsto \mathbb{R}$.

Example: Function Optimization



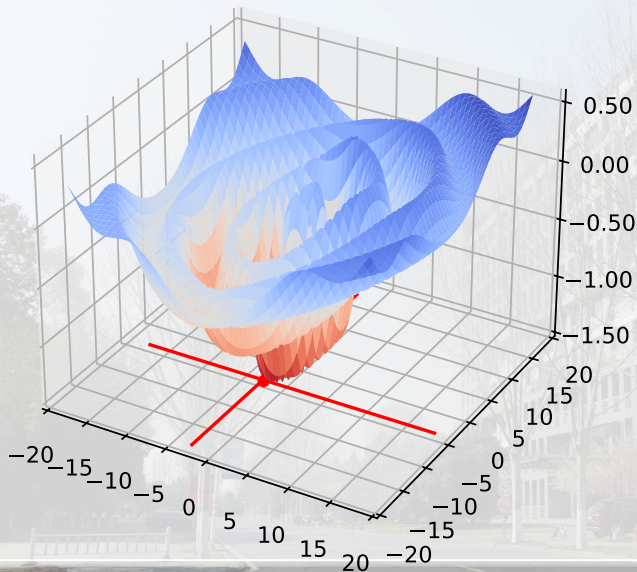
- In the field of continuous optimization, we could literally try to find the **minimum** of a mathematical function $f : \mathbb{R}^n \mapsto \mathbb{R}$.
- The search space $\mathbb{X}$ would then be the n -dimensional real vectors, i.e., $\mathbb{X} = \mathbb{R}^n$.

Example: Function Optimization



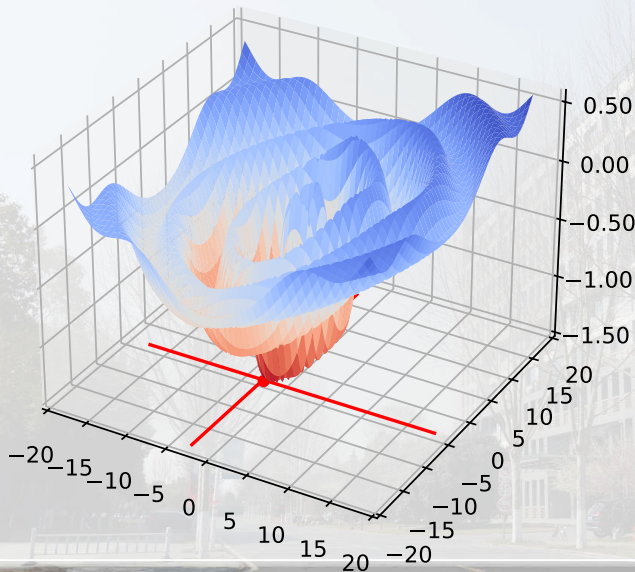
- In the field of continuous optimization, we could literally try to find the **minimum** of a mathematical function $f : \mathbb{R}^n \mapsto \mathbb{R}$.
- The search space $\mathbb{X}$ would then be the n -dimensional real vectors, i.e., $\mathbb{X} = \mathbb{R}^n$.
- The objective function would, well, be the function f .

Example: Function Optimization



- In the field of continuous optimization, we could literally try to find the **minimum** of a mathematical function $f : \mathbb{R}^n \mapsto \mathbb{R}$.
- The search space $\mathbb{X}$ would then be the n -dimensional real vectors, i.e., $\mathbb{X} = \mathbb{R}^n$.
- The objective function would, well, be the function f .
- The optimal solution $x^* \in \mathbb{X}$ is the minimum of f .

Example: Function Optimization



- In the field of continuous optimization, we could literally try to find the **minimum** of a mathematical function $f : \mathbb{R}^n \mapsto \mathbb{R}$.
- The search space $\mathbb{X}$ would then be the n -dimensional real vectors, i.e., $\mathbb{X} = \mathbb{R}^n$.
- The objective function would, well, be the function f .
- The optimal solution $x^* \in \mathbb{X}$ is the minimum of f .
- **This is *not* what I am working on, though.**

Example: Traveling Salesperson Problem

- In the Traveling Salesperson Problem (TSP)^{2,34,47,49,80,88}, the goal is to find the shortest round-trip tour through a set of n cities.



Example: Traveling Salesperson Problem

- In the Traveling Salesperson Problem (TSP)^{2,34,47,49,80,88}, the goal is to find the shortest round-trip tour through a set of n cities.
- The search space $\mathbb{X}$ thus is the set of all possible round-trip tours through these n cities, usually specified as permutations of the first n natural numbers.



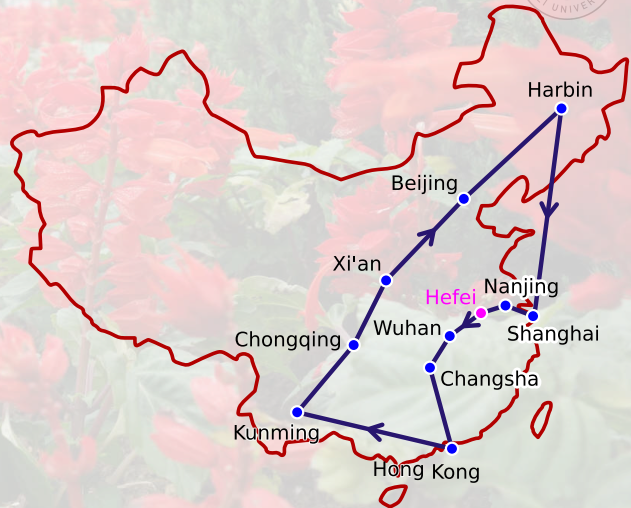
Example: Traveling Salesperson Problem

- In the Traveling Salesperson Problem (TSP)^{2,34,47,49,80,88}, the goal is to find the shortest round-trip tour through a set of n cities.
- The search space $\mathbb{X}$ thus is the set of all possible round-trip tours through these n cities, usually specified as permutations of the first n natural numbers.
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$, subject to minimization, is the length of the tour.



Example: Traveling Salesperson Problem

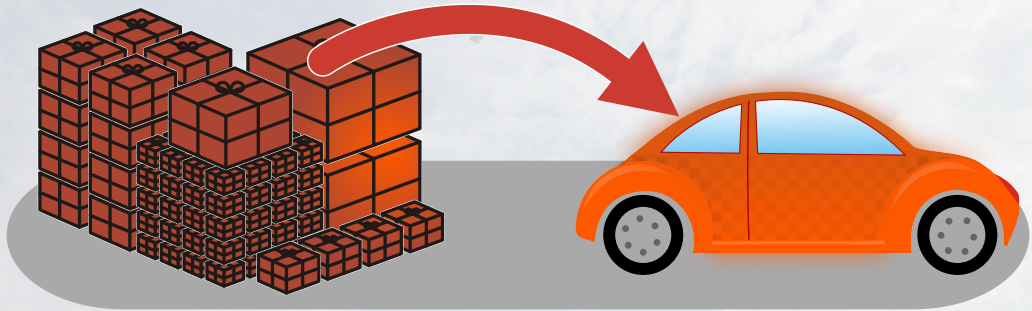
- In the Traveling Salesperson Problem (TSP)^{2,34,47,49,80,88}, the goal is to find the shortest round-trip tour through a set of n cities.
- The search space $\mathbb{X}$ thus is the set of all possible round-trip tours through these n cities, usually specified as permutations of the first n natural numbers.
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$, subject to minimization, is the length of the tour.
- The optimal solution $x^* \in \mathbb{X}$ is the shortest possible tour.



Example: Bin Packing Problem



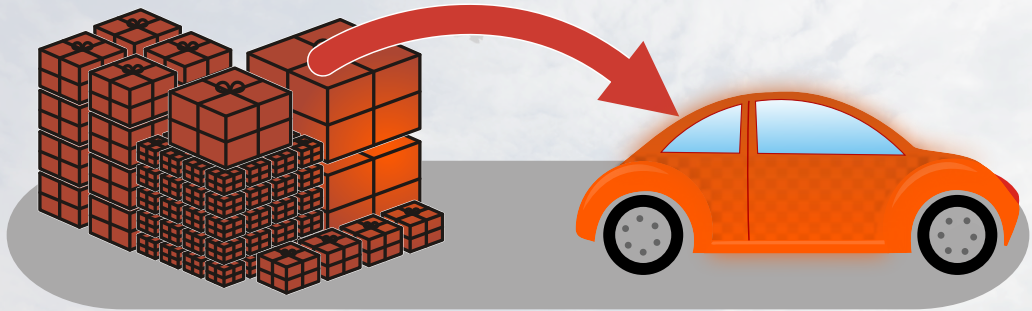
- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{94,95}.



Example: Bin Packing Problem



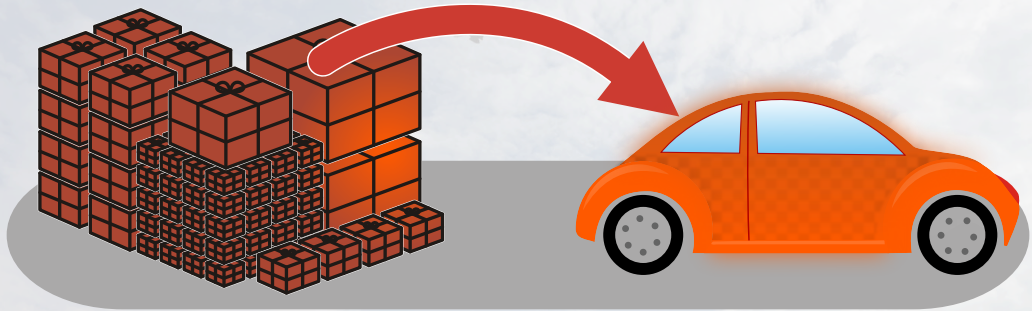
- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{94,95}.
- The $\mathbb{X}$ comprises all possible packing orders of the n objects.



Example: Bin Packing Problem



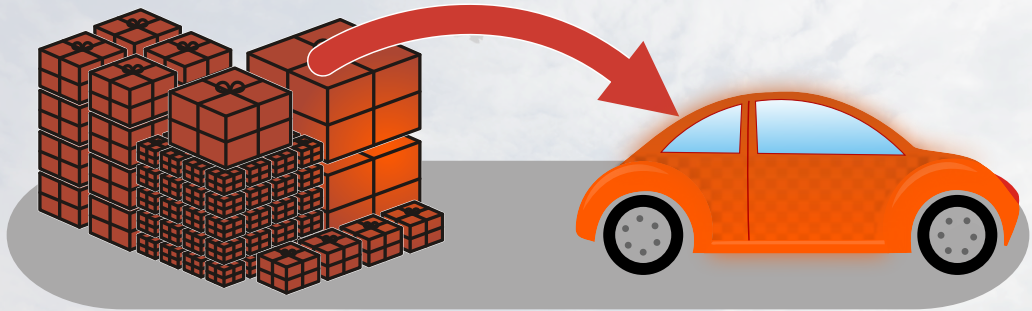
- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{94,95}.
- The $\mathbb{X}$ comprises all possible packing orders of the n objects.
- The objective function f is the number of bins needed by a given packing order.



Example: Bin Packing Problem



- The goal of the Bin Packing Problem is to pack n objects, each having a specific size, into as few bins (also of a given size) as possible^{94,95}.
- The $\mathbb{X}$ comprises all possible packing orders of the n objects.
- The objective function f is the number of bins needed by a given packing order.
- The optimum x^* is the packing order requiring the fewest bins.



Optimization is Hard



- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.



Optimization is Hard

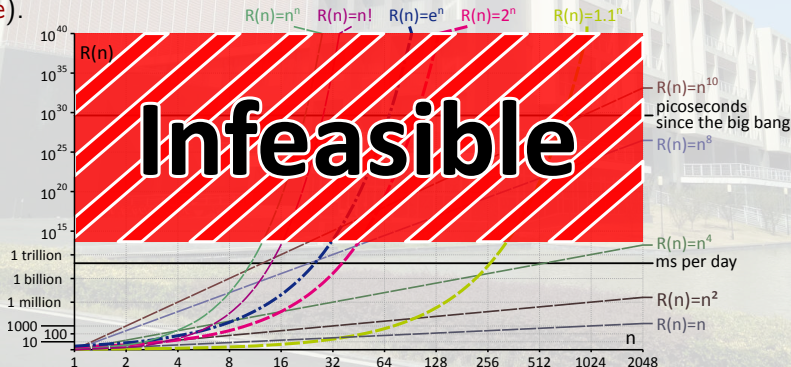


- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.
- Currently, there is no algorithm that can **guarantee** to find the optimal solution of **every instance** of a given $\mathcal{NP}$ -hard problem in a runtime that is not longer than polynomial in the size of the problem (i.e., existing algorithms may need exponential runtime in the **worst case**).

Optimization is Hard



- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.
- Currently, there is no algorithm that can **guarantee** to find the optimal solution of **every instance** of a given $\mathcal{NP}$ -hard problem in a runtime that is not longer than polynomial in the size of the problem (i.e., existing algorithms may need exponential runtime in the **worst case**).



Optimization is Hard



- Finding the globally optimal solution x^* from the set of all possible solutions $\mathbb{X}$ is often an $\mathcal{NP}$ -hard problem.
- Currently, there is no algorithm that can **guarantee** to find the optimal solution of **every instance** of a given $\mathcal{NP}$ -hard problem in a runtime that is not longer than polynomial in the size of the problem (i.e., existing algorithms may need exponential runtime in the **worst case**).
- In other words, if we want to guarantee to find the best possible solution x^* for all possible instances of a problem, we often cannot really be much faster than testing all possible candidate solutions $x \in \mathbb{X}$ in the **worst case**.

Metaheuristics



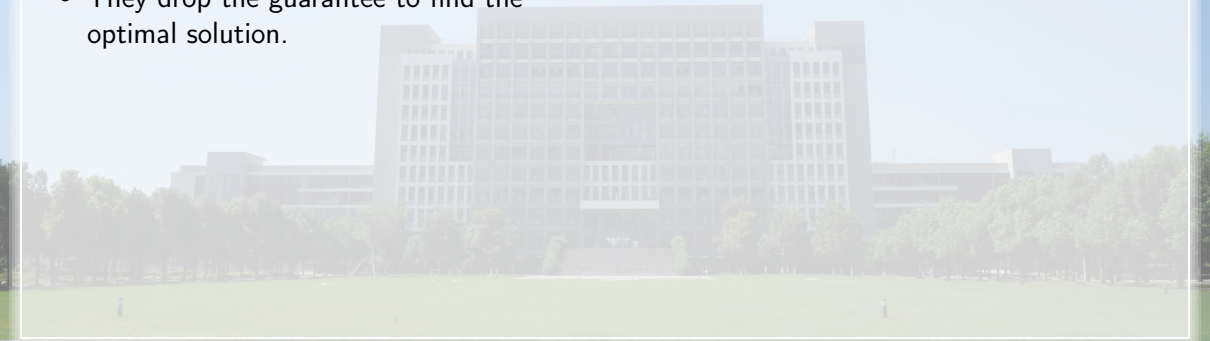
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.



Metaheuristics



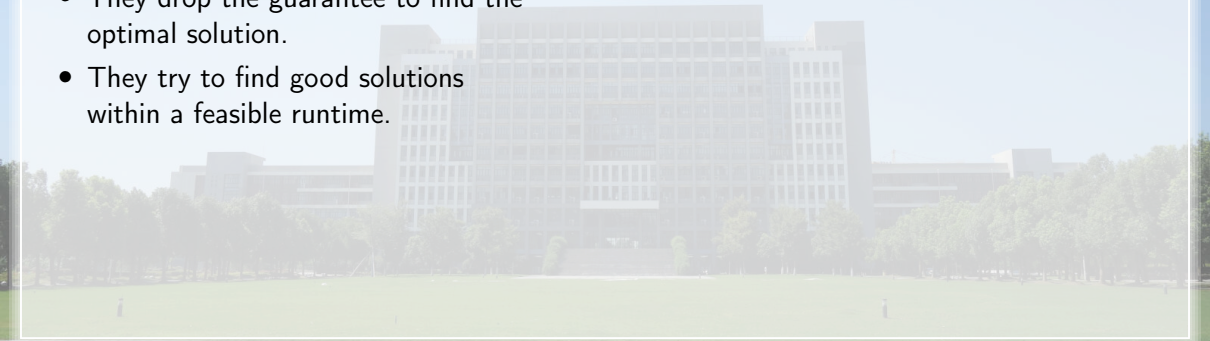
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.

A large orange circular arrow pointing clockwise, with a text box at the top.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

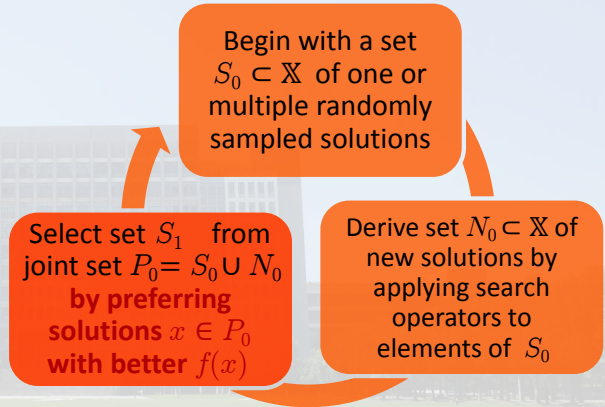
Derive set $N_0 \subset \mathbb{X}$ of new solutions by applying search operators to elements of S_0



Metaheuristics



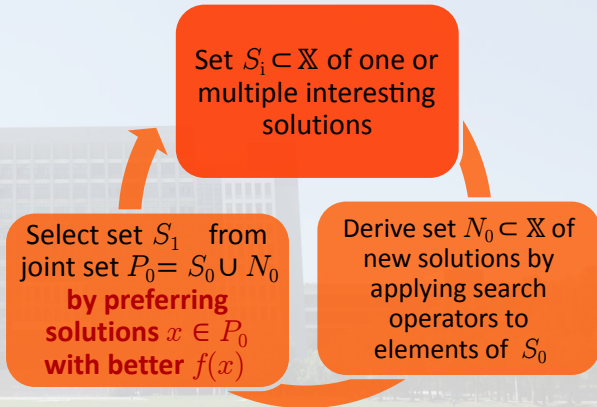
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



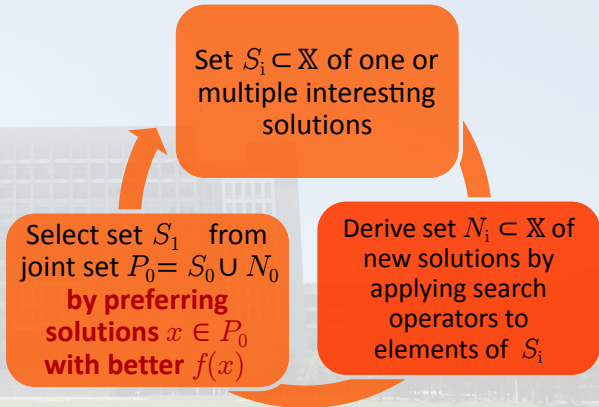
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



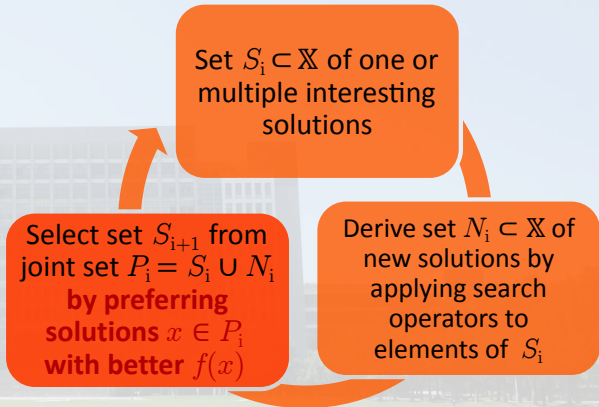
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



RLS and the $(1 + 1)$ EA

- stochastic local search (SLS) with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.



RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)
 randomly sample x_c from $\mathbb{X}$;

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)
 randomly sample x_c from $\mathbb{X}$; $y_c \leftarrow f(x_c)$;

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

Simulated Annealing

- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.



Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- Let's start with RLS/(1 + 1) EA and modify it to become SA.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
    if  $y_n \leq y_c$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
    if  $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

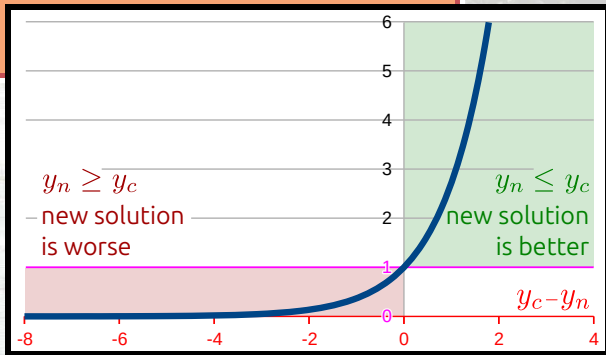
Simulated Annealing

- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

```
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

```
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then
```



Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
    if  $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then  $\triangleright$  always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.

procedure SA($f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$)

randomly sample x_c from $\mathbb{X}$; $y_c \leftarrow f(x_c)$;

while $\neg$ terminate **do**

$x_n \leftarrow \text{move}(x_c)$; $y_n \leftarrow f(x_n)$;

if $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$ **then** $\triangleright$ always true if $y_n \leq y_c$
 $x_c \leftarrow x_n$; $y_c \leftarrow y_n$;

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
     $T \leftarrow T_0$  ;  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then  $\triangleright$  always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
     $T \leftarrow T_0$  ;  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0$  ;  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;
```

Simulated Annealing



- Simulated Annealing (SA)⁴⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  return  $x_B, y_B$ 
```

Metaheuristic Optimization

- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.



Metaheuristic Optimization

- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.



Metaheuristic Optimization



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.

Metaheuristic Optimization



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

Metaheuristic Optimization



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.

A large orange circular arrow diagram, with the text box at the top and the arrow pointing clockwise, indicating a cycle.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

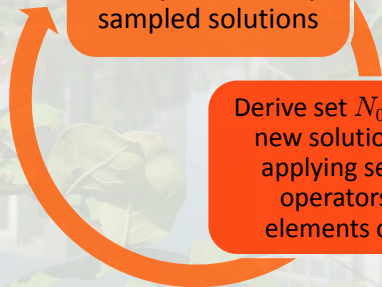
Metaheuristic Optimization



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

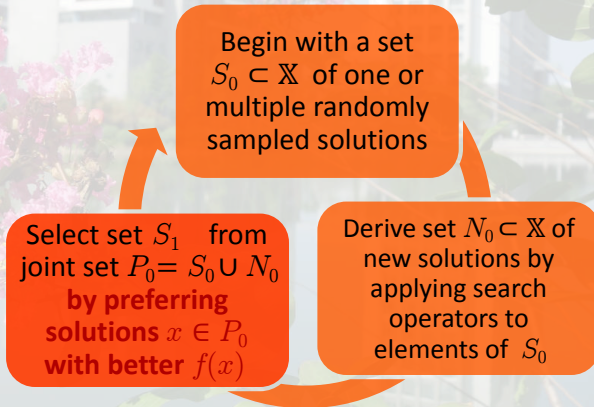
Derive set $N_0 \subset \mathbb{X}$ of new solutions by applying search operators to elements of S_0





Metaheuristic Optimization

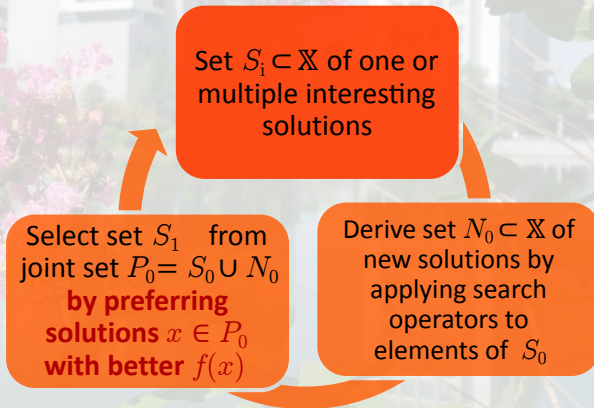
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.





Metaheuristic Optimization

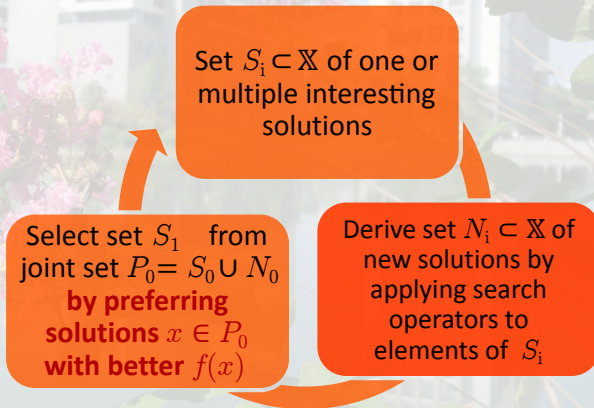
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.





Metaheuristic Optimization

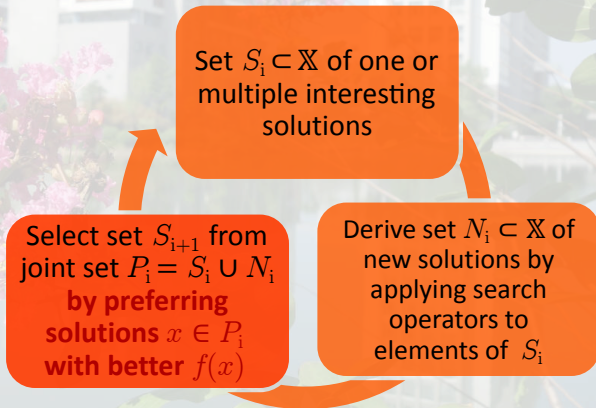
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.





Metaheuristic Optimization

- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



The (1 + 1) EA and RLS

- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.



The $(1 + 1)$ EA and RLS



- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)

The $(1 + 1)$ EA and RLS



- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)
 randomly sample x_c from $\mathbb{X}$;

The $(1 + 1)$ EA and RLS



- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

The $(1 + 1)$ EA and RLS



- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do
```

The $(1 + 1)$ EA and RLS



- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;
```

The $(1 + 1)$ EA and RLS



- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```



The $(1 + 1)$ EA and RLS

- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then
```



The $(1 + 1)$ EA and RLS

- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;
```



The $(1 + 1)$ EA and RLS

- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```



The $(1 + 1)$ EA and RLS

- Local search with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according the same pattern (and differ only in their unary search operator *move*)^{10,22}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

Research Field



- My students and I together work on solving discrete or combinatorial problems with metaheuristic algorithms.

Research Field



- My students and I together work on solving discrete or combinatorial problems with metaheuristic algorithms.
- We investigate classical hard problems that have discrete search spaces, such as bit strings or permutations.

Research Field



- My students and I together work on solving discrete or combinatorial problems with metaheuristic algorithms.
- We investigate classical hard problems that have discrete search spaces, such as bit strings or permutations.
- Here, the objective functions take on only natural numbers in $\mathbb{N}_0$.

Research Field



- My students and I together work on solving discrete or combinatorial problems with metaheuristic algorithms.
- We investigate classical hard problems that have discrete search spaces, such as bit strings or permutations.
- Here, the objective functions take on only natural numbers in $\mathbb{N}_0$.
- We try to enable metaheuristic algorithms to find better solutions for these problems.



Research Direction: Frequency Fitness Assignment



Metaheuristic Optimization Algorithms



- The most fundamental concept in metaheuristic optimization is

If you keep good solutions and modify them, you are likely to get better solutions.

Metaheuristic Optimization Algorithms



- The most fundamental concept in metaheuristic optimization is

If you keep good solutions and modify them, you are likely to get better solutions.

If you keep accepting better and better solutions, you will get really good solutions eventually.

Metaheuristic Optimization Algorithms



- The most fundamental concept in metaheuristic optimization is

If you keep good solutions and modify them, you are likely to get better solutions.

If you keep accepting better and better solutions, you will get really good solutions eventually.

- Algorithms like random sampling or exhaustive enumeration that do not at least statistically prefer better solutions have extremely bad performance.

Metaheuristic Optimization Algorithms



- The most fundamental concept in metaheuristic optimization is

If you keep good solutions and modify them, you are likely to get better solutions.

If you keep accepting better and better solutions, you will get really good solutions eventually.

- Algorithms like random sampling or exhaustive enumeration that do not at least statistically prefer better solutions have extremely bad performance.
- We **challenge** this principle.

Metaheuristic Optimization Algorithms



- The most fundamental concept in metaheuristic optimization is

If you keep good solutions and modify them, you are likely to get better solutions.

If you keep accepting better and better solutions, you will get really good solutions eventually.

- Algorithms like random sampling or exhaustive enumeration that do not at least statistically prefer better solutions have extremely bad performance.
- We **challenge** this principle.
- Our Frequency Fitness Assignment (FFA) does **not** prefer better solutions ... yet it can improve the performance of existing algorithms in several cases!

FFA: Idea



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.

FFA: Idea



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.

FFA: Idea



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.

FFA: Idea



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.
- The table H is initially filled with zeros.

FFA: Idea



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.
- The table H is initially filled with zeros.
- Before the selection step of the algorithm, $H[f(P_i[j])]$ for all $j \in 1..|P_i|$ is incremented by 1.



- Frequency Fitness Assignment (FFA) is a module that can be plugged into different existing algorithms.
- It changes the way the algorithm selects the interesting solutions S_{i+1} from the sets $P_i = S_i \cup N_i$.
- It therefore maintains a table H with the encounter frequency of each objective value in the selection decisions.
- The table H is initially filled with zeros.
- Before the selection step of the algorithm, $H[f(P_i[j])]$ for all $j \in 1..|P_i|$ is incremented by 1.
- Then, the frequencies $H[f(P_i[j])]$ replace the objective values $f(P_i[j])$ in the actual selection decisions.

FFA: (1+1) EA and (1+1) FEA



- Let's plug FFA into the (1 + 1) EA and obtain the (1 + 1) EA with FFA ((1 + 1) FEA).

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- We start with the (1 + 1) EA.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{N}$ )
```

```
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

```
  while  $\neg$  terminate do
```

```
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

```
    if  $y_n \leq y_c$  then
```

```
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

```
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- We begin by initializing the frequency table H with all zeros.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
  
    if  $y_n \leq y_c$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- *Before* the selection decision, we increment the frequency values of the objective values of all current solutions.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $y_n \leq y_c$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Now the frequency values replace the objective values in the selection decisions.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Since we may now lose the best-so-far solution, we need to track it in additional variables.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Since we may now lose the best-so-far solution, we need to track it in additional variables.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  return  $x_c, y_c$ 
```

FFA: (1+1) EA and (1+1) FEA



- Since we may now lose the best-so-far solution, we need to track it in additional variables.
- ... which are then the return values of the (1 + 1) FEA.

```
procedure (1 + 1) EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```

```
procedure (1 + 1) FEA( $f : \mathbb{X} \mapsto \mathbb{N}$ )  
   $H \leftarrow (0, 0, \dots, 0)$ ;  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $H[y_c] \leftarrow H[y_c] + 1$ ;  $H[y_n] \leftarrow H[y_n] + 1$ ;  
    if  $H[y_n] \leq H[y_c]$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;  
  return  $x_B, y_B$ 
```

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.



FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.



FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.
- Instead, solutions with less-frequent objective values are preferred.

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.
- Instead, solutions with less-frequent objective values are preferred.
- Algorithms using FFA are invariant under all injective transformations of the objective function value.

FFA: What does this do?



- The rating $H[f(x)]$ of a solution x depends only on how often solutions x' with $f(x') = f(x)$ have previously been seen in the optimization process.
- Static optimization problems become dynamic, because frequency fitness H changes over time.
- Solutions get less attractive the more often their corresponding objective values have been seen. This also holds for local optima. . .
- Solutions with better objective values are no longer preferred over such with worse objective value.
- Instead, solutions with less-frequent objective values are preferred.
- Algorithms using FFA are invariant under all injective transformations of the objective function value.
- They are less likely to get stuck at local optima, which is a problem for, e.g., the $(1 + 1)$ EA.



Our Results





snippets of [84, 86, 87]

226

IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 18, NO. 2, APRIL 2014

Frequency Fitness Assignment

Thomas Weise, *Member, IEEE*, Mingxu Wan, Pu Wang, *Member, IEEE*, Ke Tang, *Senior Member, IEEE*,
Alexandre Devert, and Xin Yao, *Fellow, IEEE*

Abstract—Metaheuristic optimization procedures such as evolutionary algorithms are usually driven by an objective function that rates the quality of a candidate solution. However, it is not clear in practice whether an objective function adequately rewards intermediate solutions on the path to the global optimum and it may exhibit deceptiveness, epistasis, neutrality, ruggedness, and a lack of causality. In this paper, we introduce the frequency fitness ff , subject to minimization, which rates how often solutions with the same objective value have been discovered so far. The ideas behind this method are that good solutions are difficult to find and that if an algorithm gets stuck at a local optimum, the frequency of the objective values of the surrounding solutions will increase over time, which will eventually allow it to leave that region again. We substitute a frequency fitness assignment process (FFA) for the objective function into several different optimization algorithms. We conduct a comprehensive set of experiments: the synthesis of algorithms with genetic programming (GP), the solution of MAX-SAT problems with genetic algorithms, classification with Memetic Genetic Programming, and numerical optimization with a (1+1) Evolution Strategy, to verify the utility of FFA. Given that they have no access to the original objective function at all, it is surprising that for some problems (e.g., the algorithm synthesis task) the FFA-based algorithm variants perform significantly better. However, this cannot be guaranteed for all tested problems. Thus, we also analyze scenarios where algorithms using FFA do not perform better or perform even worse than with the original objective functions.

Index Terms—Combinatorial optimization, diversity, fitness assignment, frequency, genetic programming (GP), numerical optimization.

I. INTRODUCTION

SINGLE-OBJECTIVE optimization is a process with the goal of finding (in all) i.e., (local) solutions a

from within a space $\mathcal{X}$ of possible solutions. An objective function f serves as quality measure guiding the search. Black-box metaheuristic approaches are methods that only require such an objective function and search operations to solve an optimization problem without any further insight into their structure. The most prominent family of these methods are evolutionary algorithms, which have wide applications ranging from engineering, planning and scheduling, numerical optimization, to even program synthesis.

Most metaheuristic optimization methods start with a randomly generated set of candidate solutions. New points in the search space are derived by modifying or combining promising existing solutions. Promising here means having a better objective value than the other points visited so far, maybe combined with some considerations about diversity. The rationale is that in the ideal case, solutions that have better objective values should be closer to the global optimum or, at least, may have even better solutions in their vicinity.

The principle of tending to choose areas of the solution space for sampling where points with better objective values have previously been discovered is one of the most universally applied ideas in black-box optimization. Lehman and Stanley [1] argued that increasing fitness does not always reveal the best path through the search space. Building on their work, we believe that there is at least one other fundamental principle inherent to non-trivial optimization problems: be exploited to solve them: good solutions (and hence, objective values) are hard to find.

If a search or optimization process is stuck in a space, it would expect a frequency



Frequency Fitness

Thomas Weise, *Member, IEEE*, Mingxu Wan, Pu Wang, Meixiang Zhang, Alexandre Devert, and Xin Yao

Abstract—Metaheuristic optimization procedures such as evolutionary algorithms are usually driven by an objective function that rates the quality of a candidate solution. However, it is not clear in practice whether an objective function adequately rewards intermediate solutions on the path to the global optimum and it may exhibit deceptiveness, epistasis, neutrality, ruggedness, and a lack of causality. In this paper, we introduce the frequency fitness ff , subject to minimization, which rates how often solutions with the same objective value have been discovered so far. The ideas behind this method are that good solutions are difficult to find and that if an algorithm gets stuck at a local optimum, the frequency of the objective values of the surrounding solutions will increase over time, which will eventually allow it to leave that region again. We substitute a frequency fitness assignment process (FFA) for the objective function into several different optimization algorithms. We conduct a comprehensive set of experiments: the synthesis of algorithms with genetic programming (GP), the solution of MAX-SAT problems with genetic algorithms, classification with Memetic Genetic Programming, and numerical optimization with a (1+1) Evolution Strategy, to verify the utility of FFA. Given that they have no access to the original objective function at all, it is surprising that for some problems (e.g., the algorithm synthesis task) the FFA-based algorithm variants perform significantly better. However, this cannot be guaranteed for all tested problems. Thus, we also analyze scenarios where algorithms using FFA do not perform better or perform even worse than with the original objective functions.

Index Terms—Combinatorial optimization, diversity, fitness assignment, frequency, genetic programming (GP), numerical optimization.

I. INTRODUCTION

SINGLE-OBJECTIVE optimization is a process with the goal of finding (in all) local solutions.

from a function f that rates the quality of a candidate solution. However, it is not clear in practice whether an objective function adequately rewards intermediate solutions on the path to the global optimum and it may exhibit deceptiveness, epistasis, neutrality, ruggedness, and a lack of causality. In this paper, we introduce the frequency fitness ff , subject to minimization, which rates how often solutions with the same objective value have been discovered so far.

The ideas behind this method are that good solutions are difficult to find and that if an algorithm gets stuck at a local optimum, the frequency of the objective values of the surrounding solutions will increase over time, which will eventually allow it to leave that region again.

We substitute a frequency fitness assignment process (FFA) for the objective function into several different optimization algorithms. We conduct a comprehensive set of experiments: the synthesis of algorithms with genetic programming (GP), the solution of MAX-SAT problems with genetic algorithms, classification with Memetic Genetic Programming, and numerical optimization with a (1+1) Evolution Strategy, to verify the utility of FFA.

Given that they have no access to the original objective function at all, it is surprising that for some problems (e.g., the algorithm synthesis task) the FFA-based algorithm variants perform significantly better. However, this cannot be guaranteed for all tested problems. Thus, we also analyze scenarios where algorithms using FFA do not perform better or perform even worse than with the original objective functions.

Frequency Fitness Assignment: Making Optimization Algorithms Invariant Under Bijective Transformations of the Objective Function Value

Thomas Weise[✉], *Member, IEEE*, Zhize Wu, Xinlu Li, and Yan Chen

Abstract—Under frequency fitness assignment (FFA), the fitness corresponding to an objective value is its encounter frequency in fitness assignment steps and is subject to minimization. FFA renders optimization processes invariant under bijective transformations of the objective function value. On TwoMax, Jump, and Trap functions of dimension s , the classical (1+1)-EA with standard mutation at rate $1/s$ can have expected runtimes exponential in s . In our experiments, a (1+1)-FEA, the same algorithm but using FFA, exhibits mean runtimes that seem to scale as $s^2 \ln s$. Since Jump and Trap are bijective transformations of OneMax, it behaves identical on all three. On OneMax, LeadingOnes, and Plateau problems, it seems to be slower than the (1+1)-EA by a factor linear in s . The (1+1)-FEA performs much better than the (1+1)-EA on W-Model and MaxSat instances. We further verify the bijection invariance by applying the Md5 checksum computation as transformation to some of the above problems and yield the same behaviors. Finally, we show that FFA can improve the performance of a memetic algorithm for job shop scheduling.

Index Terms—(1+1)-EA, evolutionary algorithm (EA), fitness assignment, job shop scheduling, memetic algorithm (MA), optimization, transformation.

which are then the basis for selection. Frequency fitness assignment (FFA) [1], [2] was developed to enable algorithms to escape from local optima. In FFA, the fitness corresponding to an objective value is its encounter frequency so far in fitness assignment steps and is subject to minimization. As we discuss in detail in Section II, FFA turns a static optimization problem into a dynamic one where objective values that are often encountered will receive worse and worse fitness.

In this article, we uncover a so-far unexplored property of FFA: it is invariant under any bijective transformation of the objective function values. This is the strongest invariance known to us and encompasses all order-preserving mappings. Other examples for bijective transformations include the negation, permutation, or even encryption of the objective values. According to [3], invariance extends performance observed on a single function to an ensemble of associated instances, but is, it generalizes from classical optimization to the fully dynamic case.

Three Core Works

snippets of [84, 86, 87]



226

IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 18, NO. 2, APRIL 2014

Frequency Fitness

Thomas Weise, Member, IEEE, Mingxu Wan, Pu Wang, Mengxiang Wang, Alexandre Devert, and Xin Yao

Abstract—Metaheuristic optimization procedures such as evolutionary algorithms are usually driven by an objective function that rates the quality of a candidate solution. However, it is not clear in practice whether an objective function adequately rewards intermediate solutions on the path to the global optimum and it may exhibit deceptiveness, epistasis, neutrality, ruggedness, and a lack of causality. In this paper, we introduce the frequency fitness FF , subject to minimization, which rates how often solutions with the same objective value have been discovered so far. The ideas behind this method are that good solutions are difficult to find and that if an algorithm gets stuck at a local optimum, the frequency of the objective values of the surrounding solutions will increase over time, which will eventually allow it to leave that region again. We substitute a frequency fitness assignment process (FFA) for the objective function into several different optimization algorithms. We conduct a comprehensive set of experiments: the synthesis of algorithms with genetic programming (GP), the solution of MAX-SAT problems with genetic algorithms, classification with Memetic Genetic Programming, and numerical optimization with a (1+1) Evolution Strategy, to verify the utility of FFA. Given that we have no access to the original objective function at all, it is surprising that for some problems (e.g., the algorithm synthesis task) the FFA-based algorithm variants perform significantly better. However, this cannot be guaranteed for all tested problems. Thus, we also analyze scenarios where algorithms using FFA do not perform better or perform even worse than with the original objective functions.

Index Terms—Combinatorial optimization, diversity, fitness assignment, frequency, genetic programming (GP), numerical optimization.

I. INTRODUCTION

SINGLE-OBJECTIVE optimization is a process with the goal of finding a (or all) local (or global) optimum of a function.

from a metaheuristic optimization procedure, the frequency fitness assignment process (FFA) is a metaheuristic optimization procedure that rates the quality of a candidate solution. However, it is not clear in practice whether an objective function adequately rewards intermediate solutions on the path to the global optimum and it may exhibit deceptiveness, epistasis, neutrality, ruggedness, and a lack of causality. In this paper, we introduce the frequency fitness FF , subject to minimization, which rates how often solutions with the same objective value have been discovered so far. The ideas behind this method are that good solutions are difficult to find and that if an algorithm gets stuck at a local optimum, the frequency of the objective values of the surrounding solutions will increase over time, which will eventually allow it to leave that region again. We substitute a frequency fitness assignment process (FFA) for the objective function into several different optimization algorithms. We conduct a comprehensive set of experiments: the synthesis of algorithms with genetic programming (GP), the solution of MAX-SAT problems with genetic algorithms, classification with Memetic Genetic Programming, and numerical optimization with a (1+1) Evolution Strategy, to verify the utility of FFA. Given that we have no access to the original objective function at all, it is surprising that for some problems (e.g., the algorithm synthesis task) the FFA-based algorithm variants perform significantly better. However, this cannot be guaranteed for all tested problems. Thus, we also analyze scenarios where algorithms using FFA do not perform better or perform even worse than with the original objective functions.

Index Terms—Combinatorial optimization, diversity, fitness assignment, frequency, genetic programming (GP), numerical optimization.

I. INTRODUCTION

SINGLE-OBJECTIVE optimization is a process with the goal of finding a (or all) local (or global) optimum of a function.

IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 25, NO. 2, APRIL 2021

Frequency Fitness Assignment Method for Bijective Transformation of Objective Functions

Thomas Weise[✉], Member, IEEE, Zhenyuan Wang, and Mingxu Wan

Abstract—Under frequency fitness assignment (FFA), the fitness corresponding to an objective value is its encounter frequency in fitness assignment steps and is subject to minimization. FFA renders optimization processes invariant under bijective transformations of the objective function value. On TwoMax, Jump, and Trap functions of dimension s , the classical (1+1)-EA with standard mutation at rate $1/s$ can have expected runtimes exponential in s . In our experiments, a (1+1)-FEA, the same algorithm but using FFA, exhibits mean runtimes that seem to scale as s^2 . In s . Since Jump and Trap are bijective transformations of OneMax, it behaves identical on all three. On OneMax, LeadingOnes, and Plateau problems, it seems to be slower than the (1+1)-EA by a factor linear in s . The (1+1)-FEA performs much better than the (1+1)-EA on W-Model and MaxSat instances. We further verify the bijection invariance by applying the Md5 checksum computation as transformation to some of the above problems and yield the same behaviors. Finally, we show that FFA can improve the performance of a memetic algorithm for job shop scheduling.

Index Terms—(1+1)-EA, evolutionary algorithm (EA), fitness assignment (FFA), job shop scheduling, mutation rate, OneMax, Plateau, TwoMax, W-model benchmark.

980

307

IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 27, NO. 4, AUGUST 2023

Frequency Fitness Assignment: Optimization Without Bias for Good Solutions Can Be Efficient

Thomas Weise[✉], Zhize Wu, Xinlu Li, Yan Chen, and Jörg Lässig

Abstract—A fitness assignment process transforms the features (such as the objective value) of a candidate solution to a scalar fitness, which then is the basis for selection. Under frequency fitness assignment (FFA), the fitness corresponding to an objective value is its encounter frequency in selection steps and is subject to minimization. FFA creates algorithms that are not biased toward better solutions and are invariant under all injective transformations of the objective function value. We investigate the impact of FFA on the performance of two theory inspired, state-of-the-art evolutionary algorithms, the Greedy (2+1) GA and the self-adjusting (1+(λ , λ)) GA. FFA improves their performance significantly on some problems that are hard for them. In our experiments, one FFA-based algorithm exhibited mean runtimes that appear to be polynomial on the theory-based benchmark problems in our study, including traps, jumps, and plateaus. We propose two hybrid approaches that use both direct and FFA-based optimization and find that they perform well. All FFA-based algorithms also perform better on satisfiability problems than any of the pure algorithm variants.

Index Terms—Evolutionary algorithm (EA), FFA, frequency fitness assignment (FFA), Ising problems, jump problems, linear harmonic functions, MaxSat problem, N-queens problem, OneMax, Plateau problems, satisfiability, Trap function, TwoMax, W-model benchmark.

fitness assignment (FFA), the fitness of a candidate solution is the absolute encounter frequency of its objective value so far during the optimization process [1]. Being subject to minimization, FFA drives the search away from already-discovered objective values and toward solutions with new qualities.

FFA breaks with one of the most fundamental concepts of heuristic optimization: FFA-based algorithms are not biased toward better solutions [2], i.e., they do not prefer better solutions over worse ones. They also are invariant under all injective transformations of the objective function value, which is the strongest invariance property of any nontrivial single-objective optimization algorithm [3].¹ Only random sampling, random walks, and exhaustive enumeration have similar properties and neither of them is considered to be an efficient optimization method.

One would expect that this comes at a significant performance penalty. Yet, FFA performed well in Genetic Programming tasks with their often rugged, deceptive, and highly epistatic landscapes [1], [4] and on a benchmark problem simulating such landscapes [5]. While the (1+1) EA has exponential expected runtime on problems, such as Jump, TwoMax, and Trap, the (1+1) FEA, the same algorithm using FFA, exhibits mean runtimes that are linear in s . [3]

INTRODUCTION

INDEX

Evolutionary

algorithm

fitness

assignment

method

for

bijective

transformation

of

objective

functions

value

encounter

frequency

selection

steps

minimization

drives

search

away

from

already-discovered

objective

values

toward

solutions

with

new

qualities

breaks

with

one

of

the

most

fundamental

concepts

of

heuristic

optimization

FFA-based

algorithms

are

not

biased

toward

better

solutions

they

do

not

prefer

better

solutions

over

worse

ones

they

also

are

invariant

under

all

injective

transformations

of

the

objective

function

value

which

is

the

strongest

invariance

property

of

any

nontrivial

single-objective

optimization

algorithm

Only

random

sampling

random

walks

and

exhaustive

enumeration

have

similar

properties

and

neither

of

them

is

considered

to

be

an

efficient

optimization

method

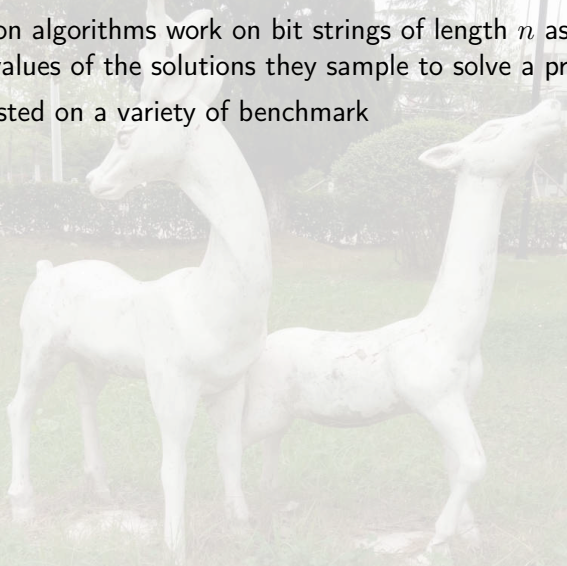
Discrete Benchmark Functions

- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.



Discrete Benchmark Functions

- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.
- They are usually tested on a variety of benchmark problems.



Discrete Benchmark Functions

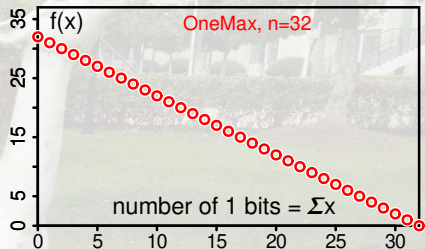


- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.
- They are usually tested on a variety of benchmark problems.
- We plug FFA into the $(1 + 1)$ EA and obtain the $(1 + 1)$ FEA.

Discrete Benchmark Functions



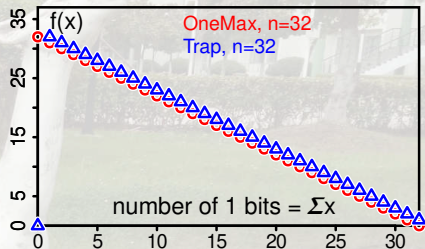
- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.
- They are usually tested on a variety of benchmark problems.
- We plug FFA into the $(1 + 1)$ EA and obtain the $(1 + 1)$ FEA.
- The $(1 + 1)$ EA can easily solve the OneMax problem, where the goal is to find a bit string of all 1s⁵⁶.



Discrete Benchmark Functions



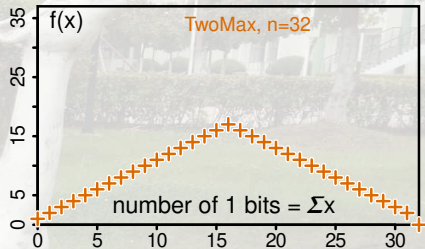
- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.
- They are usually tested on a variety of benchmark problems.
- We plug FFA into the $(1 + 1)$ EA and obtain the $(1 + 1)$ FEA.
- The $(1 + 1)$ EA can easily solve the OneMax problem, where the goal is to find a bit string of all 1s⁵⁶.
- It needs $\Theta(n^n)$ on traps²², where the optimum and the worst solution are exchanged compared to OneMax.



Discrete Benchmark Functions



- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.
- They are usually tested on a variety of benchmark problems.
- We plug FFA into the $(1 + 1)$ EA and obtain the $(1 + 1)$ FEA.
- The $(1 + 1)$ EA can easily solve the OneMax problem, where the goal is to find a bit string of all 1s⁵⁶.
- It needs $\Theta(n^n)$ on traps²², where the optimum and the worst solution are exchanged compared to OneMax, as well as on TwoMax^{27,74}, which has one local and one global optimum.



Discrete Benchmark Functions

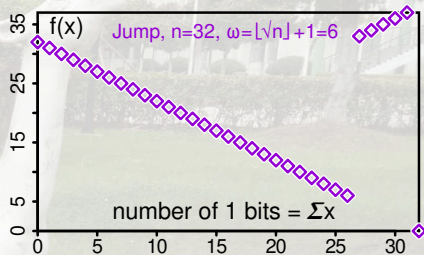


- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.
- They are usually tested on a variety of benchmark problems.
- We plug FFA into the $(1 + 1)$ EA and obtain the $(1 + 1)$ FEA.
- The $(1 + 1)$ EA can easily solve the OneMax problem, where the goal is to find a bit string of all 1s⁵⁶.
- It needs $\Theta(n^n)$ on traps²², where the optimum and the worst solution are exchanged compared to OneMax, as well as on TwoMax^{27,74}, which has one local and one global optimum.
- The $(1 + 1)$ FEA solves all three problems in polynomial time!

Discrete Benchmark Functions

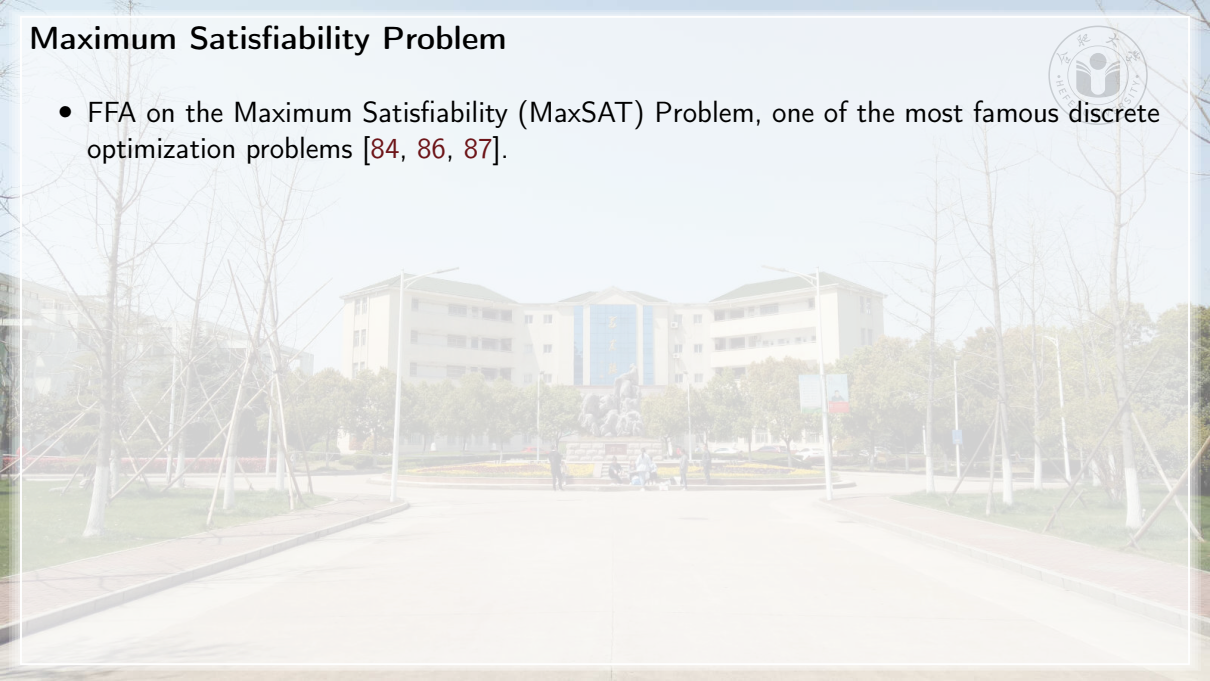


- Discrete optimization algorithms work on bit strings of length n as search space and use only the objective values of the solutions they sample to solve a problem.
- They are usually tested on a variety of benchmark problems.
- We plug FFA into the $(1 + 1)$ EA and obtain the $(1 + 1)$ FEA.
- The $(1 + 1)$ EA can easily solve the OneMax problem, where the goal is to find a bit string of all 1s⁵⁶.
- It needs $\Theta(n^n)$ on traps²², where the optimum and the worst solution are exchanged compared to OneMax, as well as on TwoMax^{27,74}, which has one local and one global optimum.
- The $(1 + 1)$ FEA solves all three problems in polynomial time as well as the Jump problem, where the $(1 + 1)$ EA also needs exponential runtime!



Maximum Satisfiability Problem

- FFA on the Maximum Satisfiability (MaxSAT) Problem, one of the most famous discrete optimization problems [84, 86, 87].



Maximum Satisfiability Problem



- FFA on the Maximum Satisfiability (MaxSAT) Problem, one of the most famous discrete optimization problems [84, 86, 87].

From Table I, we can see that **the highest number of failed runs at scale $s = 250$ of any algorithm using FFA is lower than the lowest number of failed runs of any pure algorithm at $s = 50$.** From Table II, we find that no FFA-based algorithm has a higher ERT at scale $s = 250$ than its pure variant on $s = 50$. On the scales $s \leq 75$, the FFA-based algorithms have a mean runtime which is between three and four orders of magnitude smaller than the ERT of the pure algorithms.

- Snippet of page 10 of [87] (copyright IEEE).
- Several different EAs with and without FFA

Traveling Salesperson Problem



- FFA on the Traveling Salesperson Problem (TSP): [49–51].



Traveling Salesperson Problem

- FFA on the Traveling Salesperson Problem (TSP): [49–51].
- Snippet of page 12 of [49] (copyright Springer).
- $(1 + 1)$ EA, $(1 + 1)$ FEA, Simulated Annealing (SA) w/o FFA, hybrids

In this work, we explored both the EA and SA on 56 symmetric instances from the benchmark set TSPLib (Reinelt 1991, 1995). The EA is unsuitable for this problem, but SA can find the optimum on many small and mid-sized instances. We then plug FFA into both algorithms and obtain the FEA and the FSA, respectively, which both exhibit very similar performance. **The FEA solves 27 of the instances in all of its runs, which SA can only achieve for 19 instances.** Plugging FFA into either the EA or SA thus substantially improved the number of runs in which the algorithms can find the optimum, however, both FFA-based variants suffer when the number of unique objective values is high.

• • •

Both types of **hybridization** methods significantly improve the average result quality compared to both the objective-guided and FFA-based FEA variants. **The SAFEA_A discovers the optimal solutions in more runs than any other algorithm setup in our study.** It also finds the optimum most often in most instances and delivers the best approximation quality on most instances, compared to the other algorithms.

Quadratic Assignment Problem



- FFA on the Quadratic Assignment Problem (QAP): [13, 73].



Quadratic Assignment Problem

- FFA on the Quadratic Assignment Problem (QAP): [13, 73].
- Snippet of page 7 of [13] (copyright SciTePress).
- RLS w/o FFA

We find that the **FFA-based** randomized local search **FRLS does not just find better solutions than the objective-guided RLS algorithm on the vast majority of the QAPLIB instances**, it also keeps improving its current best solution for the complete computational budget of 10^8 FEs that we assigned to the runs. With this budget, **it can discover the optimal solutions of over 58% of the QAPLIB instances**. Had we assigned a larger budget – (Liang et al., 2022; Liang et al., 2024; Weise et al., 2021b; Weise et al., 2023) use 10^{10} FEs – we would likely have seen even more instances solved.

We furthermore confirm the remarkable ability of FFA to discover very diverse solutions (at least from the perspective of the objective function). It is known that on the QAP, many solutions tend to have the same objective values (Tayarani-N. and Prügell-Bennett, 2015). Yet, on some of the instances, more than half of the objective values discovered by FRLS were unique.

Job Shop Scheduling Problem



- FFA on the Job Shop Scheduling Problem (JSSP) Problem: [18, 81, 86].

Job Shop Scheduling Problem



- FFA on the Job Shop Scheduling Problem (JSSP) Problem: [18, 81, 86].

The **end result quality delivered by (1+1)-FEA is better in average on the abz*, ft*, la*, orb*, and yn4* instance sets, both in terms of *best* and *mean*.** On swv*, the average for *mean* is better for (1+1)-FEA, while (1+1)-EA has a slight lead in *best*. The (1+1)-EA performs better on the dmμ* and ta* instances. Since these two sets are larger (holding 160 out of the 242 instances), the (1+1)-EA comes out ahead in the overall averages, but with no more than a 1.5% advantage.

- Snippet of page 10 of [81] (copyright ACM).

Algorithm Synthesis and Genetic Programming



- FFA for algorithm synthesis and Genetic Programming (GP): [83, 84].

Algorithm Synthesis and Genetic Programming



- FFA for algorithm synthesis and Genetic Programming (GP): [83, 84].

Fourth, we were able to confirm with great significance that **FFA** has a tremendous positive impact on the performance of GP, as it **can increase the success rate by 40%**.

- Snippet of page 7 of [83] (copyright IEEE).



Future Works



Future Works



- Apply FFA to other discrete and combinatorial problems.

Future Works



- Apply FFA to other discrete and combinatorial problems:
 1. Where can FFA improve the solution quality?

Future Works



- Apply FFA to other discrete and combinatorial problems:
 1. Where can FFA improve the solution quality?
 2. Where can it not do that?

Future Works



- Apply FFA to other discrete and combinatorial problems:
 1. Where can FFA improve the solution quality?
 2. Where can it not do that?
 3. Why?

Future Works



- Apply FFA to other discrete and combinatorial problems:
 1. Where can FFA improve the solution quality?
 2. Where can it not do that?
 3. Why?
- Plug FFA into more optimization algorithms.

Future Works



- Apply FFA to other discrete and combinatorial problems:
 1. Where can FFA improve the solution quality?
 2. Where can it not do that?
 3. Why?
- Plug FFA into more optimization algorithms.
- Combine “traditional” and FFA-based optimization, i.e., create hybrid algorithms, to reap the best of both worlds.



Summary



Summary



- Our students and I work on the big field of optimization.

Summary



- Our students and I work on the big field of optimization.
- We try to improve the quality of solutions that metaheuristics can deliver on discrete or combinatorial problems.

Summary



- Our students and I work on the big field of optimization.
- We try to improve the quality of solutions that metaheuristics can deliver on discrete or combinatorial problems.
- We have a very interesting and novel technique called Frequency Fitness Assignment (FFA).

Summary



- Our students and I work on the big field of optimization.
- We try to improve the quality of solutions that metaheuristics can deliver on discrete or combinatorial problems.
- We have a very interesting and novel technique called Frequency Fitness Assignment (FFA).
- We are figuring out where it really can give us better results.

Summary



- Our students and I work on the big field of optimization.
- We try to improve the quality of solutions that metaheuristics can deliver on discrete or combinatorial problems.
- We have a very interesting and novel technique called Frequency Fitness Assignment (FFA).
- We are figuring out where it really can give us better results.
- But we are not picky.

Summary



- Our students and I work on the big field of optimization.
- We try to improve the quality of solutions that metaheuristics can deliver on discrete or combinatorial problems.
- We have a very interesting and novel technique called Frequency Fitness Assignment (FFA).
- We are figuring out where it really can give us better results.
- But we are not picky. For example,
 1. on the two-dimensional bin packing problem, we found out that a simple local search can actually **outperform** the complex state-of-the-art metaheuristics and FFA could not improve the performance ... so we published this surprising result and the student graduated with it^{94,95}.

Summary



- Our students and I work on the big field of optimization.
- We try to improve the quality of solutions that metaheuristics can deliver on discrete or combinatorial problems.
- We have a very interesting and novel technique called Frequency Fitness Assignment (FFA).
- We are figuring out where it really can give us better results.
- But we are not picky. For example,
 1. on the two-dimensional bin packing problem, we found out that a simple local search can actually **outperform** the complex state-of-the-art metaheuristics and FFA could not improve the performance ... so we published this surprising result and the student graduated with it^{94,95}, and
 2. on the Traveling Tournament Problem (TTP), we also got surprisingly good results with RLS⁹⁰ and SA.



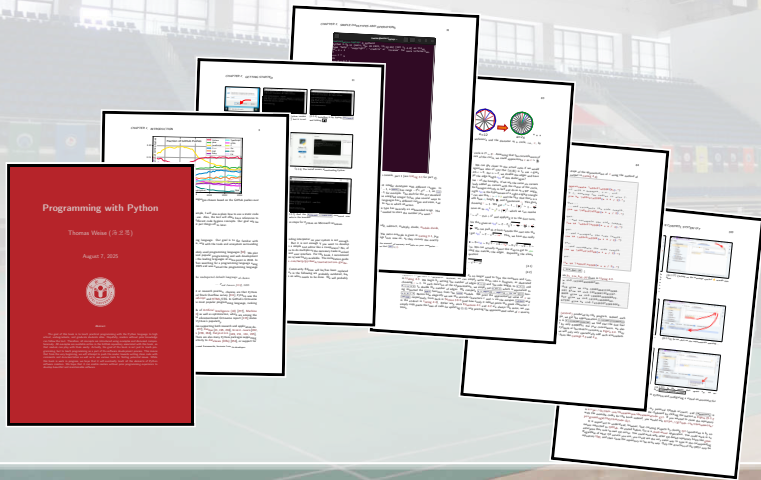
Advertisement



Programming with Python



We have a freely available course book on *Programming with Python* at <https://thomasweise.github.io/programmingWithPython>, with focus on practical software development using the Python ecosystem of tools⁷⁹.



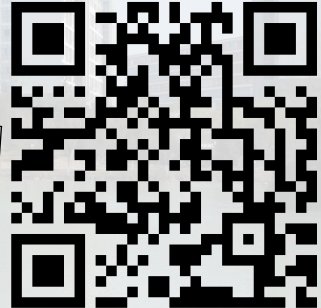
Databases



We have a freely available course book on *Databases* at <https://thomasweise.github.io/databases>, with actual practical examples using a real database management system (DBMS)⁷⁶.



ization,





谢谢你们！
Thank you!
Vielen Dank!



References I

- 
- [1] Denis Antipov and Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 117–128. ISSN: 0302-9743. ISBN: 978-3-319-99258-7. doi:10.1007/978-3-319-99259-4_10 (cit. on p. 172).
- [2] David Lee Applegate, Robert E. Bixby, Vašek Chvátal, and William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2nd ed. Vol. 17 of Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: 978-0-691-12993-8. This book also discusses the Concorde TSP solver. (Cit. on pp. 16–19, 169, 174).
- [3] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: 978-1-4181-6963-3. URL: <http://gallica.bnf.fr/ark:/12148/bpt6k994750> (visited on 2023-12-13) (cit. on p. 176).
- [4] Thomas Bäck, David B. Fogel, and Zbigniew "Zbyszek" Michalewicz, eds. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd and Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: 978-0-7503-0392-7 (cit. on p. 170).
- [5] Tim Berners-Lee. *Re: Qualifiers on Hypertext links*. . . Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: <https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt> (visited on 2025-02-05) (cit. on p. 175).
- [6] Tim Berners-Lee, Larry Masinter, and Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 1994. URL: <https://www.ietf.org/rfc/rfc1738.txt> (visited on 2025-02-05) (cit. on p. 175).
- [7] Jacek Błażewicz, Wolfgang Domschke, and Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (cit. on p. 170).
- [8] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, May 2008. ISBN: 978-0-321-35668-0 (cit. on p. 170).
- [9] Rainer E. Burkard, Eranda Çela, Panos Miltiades Pardalos, and Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (cit. on p. 172).

References II



- [10] Eduardo Carvalho Pinto and Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell University Library, Dec. 3, 2018. doi:10.48550/arXiv.1812.00493. URL: <https://arxiv.org/abs/1812.00493> (visited on 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (cit. on pp. 38–48, 72–82, 169).
- [11] Vladimír Černý. “Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm”. *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: 0022-3239. doi:10.1007/BF00940812 (cit. on p. 173).
- [12] Bo Chen, Chris N. Potts, and Gerhard J. Woeginger. “A Review of Machine Scheduling: Complexity, Algorithms and Approximability”. In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miliotopoulos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1493–1641. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_25. See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. (Cit. on pp. 170, 176).
- [13] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson, and Thomas Weise (汤卫恩). “Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem”. In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 27–37. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888600003837 (cit. on pp. 130, 131, 170–172).
- [14] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra, and Zhen Liu, eds. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sept. 1995. ISBN: 978-0-471-94059-3 (cit. on p. 170).
- [15] Stephen Arthur Cook. “The Complexity of Theorem-Proving Procedures”. In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. May 3–5, 1971, Shaker Heights, OH, USA. Ed. by Michael A. Harrison, Ranjan B. Banerji, and Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, pp. 151–158. ISBN: 978-1-4503-7464-4. doi:10.1145/800157.805047 (cit. on pp. 171, 176).
- [16] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: <https://math.uwaterloo.ca/tsp/concorde> (visited on 2026-09-26) (cit. on p. 169).

References III



- [17] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–Nov. 9, 2014. ISBN: **978-0-691-16352-9** (cit. on p. **169**).
- [18] Ege de Bruin, Sarah Louise Thomson, and Daan van den Berg. "Frequency Fitness Assignment on JSSP: A Critical Review". In: *26th European Conference on Applications of Evolutionary Computation (EvoApplications'2023), Held as Part of EvoStar'2023*. Apr. 12–14, 2023, Brno, Czech Republic. Ed. by João Correia, Stephen Smith, and Raneem Qaddoura. Vol. 13989 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2023, pp. 351–363. ISSN: **0302-9743**. ISBN: **978-3-031-30228-2**. doi:[10.1007/978-3-031-30229-9_23](https://doi.org/10.1007/978-3-031-30229-9_23). Independent reproduction of ⁸¹ (cit. on pp. **132**, **133**).
- [19] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: **978-0-13-299045-5** (cit. on p. **175**).
- [20] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: **979-8-8688-0224-9** (cit. on p. **169**).
- [21] Benjamin Doerr, Carola Doerr, and Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **0304-3975**. doi:[10.1016/J.TCS.2014.11.028](https://doi.org/10.1016/J.TCS.2014.11.028) (cit. on p. **172**).
- [22] Stefan Droste, Thomas Jansen, and Ingo Wegener. "On the Analysis of the $(1 + 1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **0304-3975**. doi:[10.1016/S0304-3975\(01\)00182-7](https://doi.org/10.1016/S0304-3975(01)00182-7) (cit. on pp. **38–48**, **72–82**, **118–125**, **169–171**, **173**).
- [23] Stefan Droste, Thomas Jansen, and Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, July 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **0304-3975**. doi:[10.1007/S00224-004-1177-Z](https://doi.org/10.1007/S00224-004-1177-Z) (cit. on p. **172**).
- [24] Kelly Easton, George L. Nemhauser, and Michael A. Trick. "The Traveling Tournament Problem Description and Benchmarks". In: *7th International Conference on Principles and Practice of Constraint Programming (CP'01)*. Nov. 26–Dec. 1, 2001, Paphos, Cyprus. Ed. by Toby Walsh. Vol. 2239 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2001, pp. 580–584. ISSN: **0302-9743**. ISBN: **978-3-540-42863-3**. doi:[10.1007/3-540-45578-7_43](https://doi.org/10.1007/3-540-45578-7_43) (cit. on p. **174**).

References IV



- [25] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: **978-1-83763-564-1** (cit. on p. **172**).
- [26] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt, and Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Win. 2009. Cambridge, MA, USA: MIT Press. ISSN: **1063-6560**. doi:**10.1162/evco.2009.17.4.17401** (cit. on p. **174**).
- [27] Tobias Friedrich, Francesco Quinzan, and Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyoto, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 293–300. ISBN: **978-1-4503-5618-3**. doi:**10.1145/3205455.3205515** (cit. on pp. **118–125, 170, 174**).
- [28] Michael R. Garey and David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: **978-0-7167-1045-5** (cit. on p. **174**).
- [29] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sum. 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:**10.1287/ijoc.1.3.190** (cit. on p. **173**).
- [30] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Win. 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:**10.1287/ijoc.2.1.4** (cit. on p. **173**).
- [31] Fred Glover and Gary A. Kochenberger, eds. *Handbook of Metaheuristics*. Vol. 57 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: **0884-8289**. ISBN: **978-0-306-48056-0**. doi:**10.1007/b101874** (cit. on p. **171**).
- [32] Fred Glover, Éric D. Taillard, and Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, Mar. 1993. London, England, UK: Springer Nature Limited. ISSN: **0254-5330**. doi:**10.1007/BF02078647**. Special issue on Tabu Search (TS). (Cit. on p. **173**).
- [33] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: <https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf> (visited on 2025-08-01) (cit. on pp. **175, 176**).

References V



- [34] Gregory Z. Gutin and Abraham P. Punnen, eds. *The Traveling Salesman Problem and its Variations*. Vol. 12 of Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, May 2002. ISSN: 1388-3011. doi:10.1007/b101971 (cit. on pp. 16–19, 174).
- [35] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: 978-1-55860-872-6. doi:10.1016/B978-1-55860-872-6.X5016-1 (cit. on pp. 171, 173).
- [36] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: 1863-7310. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on p. 172).
- [37] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on p. 168).
- [38] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. Oct. 5–7, 1982. Ed. by Joseph Allen Fisher, William J. Tracz, and Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and New York, NY, USA: Association for Computing Machinery (ACM), Oct. 1982, pp. 143–148. doi:10.5555/800036.800944. See ³⁹ (cit. on p. 160).
- [39] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dec. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1050-916X. doi:10.1145/1014194.800944. See ³⁸ (cit. on pp. 160, 173).
- [40] Scott Kirkpatrick, C. Daniel Gelatt, Jr., and Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, May 13, 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: 0036-8075. doi:10.1126/science.220.4598.671. URL: <https://www.researchgate.net/publication/6026283> (visited on 2025-08-08) (cit. on pp. 49–61, 173).
- [41] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–June 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5700. doi:10.1145/1008328.1008329 (cit. on p. 176).

References VI



- [42] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: **978-0-201-89683-1** (cit. on p. 176).
- [43] Tjalling Charles Koopmans and Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: **0012-9682**. doi:[10.2307/1907742](https://doi.org/10.2307/1907742) (cit. on p. 172).
- [44] John R. Koza. *Genetic Programming – On the Programming of Computers by Means of Natural Selection*. Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, 1993. ISBN: **978-0-262-11170-6** (cit. on p. 170).
- [45] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: **978-0-8218-2650-8** (cit. on p. 176).
- [46] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Ed. by Stephen C. Graves, Alexander Hendrik George Rinnooy Kan, and Paul H. Zipkin. Vol. IV of Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Chap. 9, pp. 445–522. ISSN: **0927-0507**. ISBN: **978-0-444-87472-6**. doi:[10.1016/S0927-0507\(05\)80189-6](https://doi.org/10.1016/S0927-0507(05)80189-6). URL: <http://alexandria.tue.nl/repository/books/339776.pdf> (visited on 2023-12-06) (cit. on pp. 170, 176).
- [47] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sept. 1985. ISSN: **0277-2698**. ISBN: **978-0-471-90413-7** (cit. on pp. 16–19, 174).
- [48] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: **1863-7310**. ISBN: **978-3-319-13071-2**. doi:[10.1007/978-3-319-13072-9](https://doi.org/10.1007/978-3-319-13072-9) (cit. on p. 172).
- [49] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson, and Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, July 2024. London, England, UK: Springer Nature Limited. ISSN: **1432-7643**. doi:[10.1007/S00500-024-09718-8](https://doi.org/10.1007/S00500-024-09718-8) (cit. on pp. 16–19, 128, 129, 170, 171, 174).

References VII



- [50] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, and Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. Dec. 4–7, 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: 978-1-6654-8769-6. doi:10.1109/SSCI51031.2022.10022296 (cit. on pp. 128, 129, 170, 171).
- [51] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thüerer, Markus Wagner, and Thomas Weise (汤卫思). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 173–180. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888800003837 (cit. on pp. 128, 129, 171).
- [52] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn, and Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/j.ejor.2005.09.032 (cit. on p. 172).
- [53] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: 978-1-4920-5627-0 (cit. on p. 170).
- [54] Mark Lutz. *Learning Python*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2025. ISBN: 978-1-0981-7130-8 (cit. on p. 172).
- [55] Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe, eds. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0000195000003837.
- [56] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 15–26. URL: <https://www.researchgate.net/publication/284805798> (visited on 2025-08-15) (cit. on pp. 118–125, 172).
- [57] Siegfried Nijssen and Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2002.806169 (cit. on p. 173).

References VIII



- [58] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: **978-1-4919-6336-4** (cit. on p. 172).
- [59] Panos Miliadis Pardalos, Ding-Zhu Du, and Ronald Lewis Graham, eds. *Handbook of Combinatorial Optimization*. 1st ed. Boston, MA, USA: Springer, 1998. ISBN: **978-1-4613-7987-4**.
- [60] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dec. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0030-364X**. doi:10.1287/opre.18.6.1225 (cit. on p. 173).
- [61] Riccardo Poli, William B. Langdon, and Nicholas Freitag McPhee. *A Field Guide to Genetic Programming*. lulu.com, 2008. ISBN: **978-1-4092-0073-4**. URL: <https://www.researchgate.net/publication/216301261> (visited on 2025-08-18) (cit. on p. 170).
- [62] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on p. 172).
- [63] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/31EC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: <https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf> (visited on 2024-06-29) (cit. on p. 169).
- [64] Sanatan Rai and George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Ed. by Christodoulos A. Floudas and Panos Miliadis Pardalos. 2nd ed. Boston, MA, USA: Springer, Sept. 2008, pp. 2675–2682. ISBN: **978-0-387-74758-3**. doi:10.1007/978-0-387-74759-0_462 (cit. on p. 176).
- [65] Soraya B. Rana and L. Darrell Whitley. "Genetic Algorithm Behavior in the MAXSAT Domain". In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. Sept. 27–30, 1998, Amsterdam, The Netherlands. Ed. by Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, pp. 785–794. ISSN: **0302-9743**. ISBN: **978-3-540-65078-2**. doi:10.1007/BFb0056920 (cit. on p. 171).
- [66] Gerhard Reinelt. "TSPLIB – A Traveling Salesman Problem Library". *ORSA Journal on Computing* 3(4):376–384, Aut. 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:10.1287/ijoc.3.4.376 (cit. on p. 174).

References IX



- [67] Gerhard Reinelt. *TSPLIB95*. Tech. rep. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–Apr. 28, 2016. URL: <https://github.com/shredderzvj/TSPLIB> (visited on 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite> of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Cit. on p. 174).
- [68] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Vol. 35 of *Forschungsergebnisse zur Informatik*. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (cit. on p. 171).
- [69] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (AIMA)*. 4th ed. Hoboken, NJ, USA: Pearson Education, Inc. ISBN: 978-1-292-40113-3. URL: <https://aima.cs.berkeley.edu> (visited on 2024-06-27) (cit. on p. 169).
- [70] Sartaj Sahni and Teofilo Francisco Gonzalez Arce. "P-complete Approximation Problems". *Journal of the ACM (JACM)* 23(3):555–565, July 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: <https://sites.cs.ucsb.edu/~teo/papers/JACM-PCom.pdf> (visited on 2026-02-21) (cit. on p. 172).
- [71] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: <https://arodes.hes-so.ch/record/8224/files/Preprint.pdf> (visited on 2026-02-20) (cit. on p. 170).
- [72] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: 978-1-80323-347-5 (cit. on p. 172).
- [73] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇), and Thomas Weise (汤卫思). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Vol. 1. Sept. 14–18, 2024, Hagenberg, Mühlkreis, Austria. Ed. by Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck. Vol. 15148 of *Lecture Notes in Computer Science (LNCS)*. Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23 (cit. on pp. 130, 131, 170, 171).

References X



- [74] Clarissa Van Hoyweghen, David Edward Goldberg, and Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. July 9–13, 2002, New York, NY, USA. Ed. by William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke, and Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, pp. 626–633. ISBN: 978-1-55860-878-8 (cit. on pp. 118–125, 174).
- [75] Kristian Verduin, Sarah Louise Thomson, and Daan van den Berg. "Too Constrained for Genetic Algorithms. Too Hard for Evolutionary Computing. The Traveling Tournament Problem". In: *15th International Joint Conference on Computational Intelligence (IJCCI'23)*. Nov. 13–15, 2023, Rome, Italy. Ed. by Niki van Stein, Francesco Marcelloni, H. K. Lam, Marie Cottrell, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2023, pp. 246–257. ISSN: 2184-3236. ISBN: 978-989-758-674-3. doi:10.5220/0012192100003595 (cit. on p. 174).
- [76] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: <https://thomasweise.github.io/databases> (visited on 2025-01-05) (cit. on pp. 153, 169).
- [77] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: <https://www.researchgate.net/publication/200622167> (visited on 2025-07-25) (cit. on pp. 4–6, 170, 171).
- [78] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: <https://thomasweise.github.io/programmingWithC> (visited on 2026-05-10) (cit. on p. 169).
- [79] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: <https://thomasweise.github.io/programmingWithPython> (visited on 2025-01-05) (cit. on pp. 152, 172).

References XI



- [80] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew "Zbyszek" Michalewicz, and Xin Yao (姚新). "Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem". *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: **1556-603X**. doi:[10.1109/MCI.2014.2326101](https://doi.org/10.1109/MCI.2014.2326101) (cit. on pp. **16–19, 174**).
- [81] Thomas Weise (汤卫思), Xinlu Li (李新路), Yan Chen (陈岩), and Zhize Wu (吴志泽). "Solving Job Shop Scheduling Problems without using a Bias for Good Solutions". In: *23rd Genetic and Evolutionary Computation Conference (GECCO'2021), Companion Volume*. July 10–14, 2021, Lille, France. Ed. by Krzysztof Krawiec. New York, NY, USA: Association for Computing Machinery (ACM), 2021, pp. 1459–1466. ISBN: **978-1-4503-8351-6**. doi:[10.1145/3449726.3463124](https://doi.org/10.1145/3449726.3463124) (cit. on pp. **132, 133, 158**).
- [82] Thomas Weise (汤卫思) and Ke Tang (唐珂). "Evolving Distributed Algorithms with Genetic Programming". *IEEE Transactions on Evolutionary Computation* 16(2):242–265, Apr. 2012. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: **1089-778X**. doi:[10.1109/TEVC.2011.2112666](https://doi.org/10.1109/TEVC.2011.2112666) (cit. on p. **170**).
- [83] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Ke Tang (唐珂), and Xin Yao (姚新). "Evolving Exact Integer Algorithms with Genetic Programming". In: *IEEE Congress on Evolutionary Computation (CEC'2014)*. July 6–11, 2014, China, Beijing (中国北京市). Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2014, pp. 1816–1823. ISBN: **978-1-4799-1488-3**. doi:[10.1109/CEC.2014.6900292](https://doi.org/10.1109/CEC.2014.6900292) (cit. on pp. **134, 135**).
- [84] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert, and Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: **1089-778X**. doi:[10.1109/TEVC.2013.2251885](https://doi.org/10.1109/TEVC.2013.2251885) (cit. on pp. **115–117, 126, 127, 134, 135, 170**).
- [85] Thomas Weise (汤卫思) and Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. July 15–19, 2023, Lisbon, Portugal. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: **979-8-4007-0120-7**. doi:[10.1145/3583133.3596306](https://doi.org/10.1145/3583133.3596306) (cit. on pp. **154, 171**).

References XII



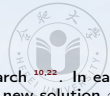
- [86] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), and Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2020.3032090 (cit. on pp. 115–117, 126, 127, 132, 133, 169–174).
- [87] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩), and Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2022.3191698 (cit. on pp. 115–117, 126, 127, 169, 170, 173, 174).
- [88] Thomas Weise (汤卫思), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂), and Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: 0925-5001. doi:10.1007/s10898-016-0417-5 (cit. on pp. 16–19, 174).
- [89] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. June 1989, Fairfax, VA, USA: George Mason University. Ed. by J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, pp. 116–123. ISBN: 978-1-55860-066-9. URL: <https://www.researchgate.net/publication/2527551> (visited on 2025-08-08) (cit. on p. 171).
- [90] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg, and Thomas Weise (汤卫思). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 38–49. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012891500003837 (cit. on pp. 144–150, 171, 174).
- [91] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: <https://www.techtarget.com/searchdatamanagement/definition/database-management-system> (visited on 2025-01-11) (cit. on p. 169).

References XIII



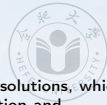
- [92] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: <https://www.ietf.org/rfc/rfc3629.txt> (visited on 2025-02-05). See Unicode and ³⁷ (cit. on p. 174).
- [93] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽), and Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, pp. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098 (cit. on pp. 169–174).
- [94] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer, and Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. July 14–18, 2024, Melbourne, VIC, Australia. Ed. by Xiaodong Li and Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (cit. on pp. 20–23, 144–150, 171).
- [95] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thürer, Tianyu Liang (梁天宇), Ming Tan (檀明), and Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (cit. on pp. 20–23, 144–150, 171).

Glossary I



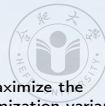
- (1 + 1) EA The (1 + 1) EA is a local search algorithm that retains the best solution x_c discovered so far during the search^{10,22}. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then the (1 + 1) EA uses a unary search operator that flips each bit independently with probability m/n , where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The (1 + 1) EA is a special case of the $(\mu + \lambda)$ evolutionary algorithm $((\mu + \lambda)$ EA) where $\mu = \lambda = 1$.
- (1 + 1) FEA The (1 + 1) EA with FFA plugged in^{86,87,93}. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where n is the dimension of the search space, distinguishing it from the unary operator in RLS with FFA (FRLS) which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.
- $(\mu + \lambda)$ EA The $(\mu + \lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu + \lambda$ solutions, from which then the best μ solutions are retained to form the parent population of the next generation. If the search space is the bit strings of length n , then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.
- AI Artificial Intelligence, see, e.g.,⁶⁹
- C is a programming language, which is very successful in system programming situations^{20,63,78}.
- Concorde is an exact solver for the TSP^{2,16,17}, which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.
- DB A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases*⁷⁶.
- DBMS A *database management system* is the software layer located between the user or application and the database (DB). The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB⁹¹.

Glossary II



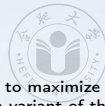
- EA** An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, respectively) ^{4,77}.
- FFA** *Frequency Fitness Assignment* (中文: 频率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values ^{84,86,87}. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the MaxSAT ^{86,87}, the TSP ^{49,50}, and the QAP ^{13,73}, amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau ^{86,87,93}. The most common realizations of FFA so far are the (1 + 1) FEA and FRLS.
- GP** Genetic Programming ^{44,61,77,82} is the application of metaheuristic optimization, usually in form of an EAs, to a search space comprised of tree datastructures. These tree datastructures often represent programs or mathematical expressions.
- Java** is another very successful programming language, with roots in the C family of languages ^{8,53}.
- JSSP** The *Job Shop Scheduling Problem* ^{7,14,46} is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are k machines and m jobs ⁷¹. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$ -complete ^{12,46}.
- Jump** The Jump problem is a benchmark problem for bit-string based black-box search ^{22,27,93}. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length n . However, the solutions representing the last ω bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the deceptive basin around the optimum that an algorithm needs to cross. An (1 + 1) EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$ ²². In experiments, the (1 + 1) FEA achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{86,87,93}.

Glossary III



- LeadingOnes** The LeadingOnes problem is a benchmark problem for bit-string based black-box search^{68,89,93}. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n \prod_{j=1}^i x_j$. Different from OneMax, the contribution to the objective value of the bit at index i thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$ ²². The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^3)$ ⁸⁶.
- MaxSAT** The goal of satisfiability problems is to find an assignment for n Boolean variables that make a given Boolean formula $F : \{0, 1\}^n \mapsto \{0, 1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT)³⁵, F is given in conjunctive normal form, i.e., the variables appear either directly or negated in m "or" clauses, which are all combined into one "and." The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting x . If $f(x) = 0$, then all clauses of F are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$ -complete¹⁵ and exhibits low epistasis but deceptiveness⁶⁵, making it hard for local searches.
- metaheuristic** A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$ ^{31,77}. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions or similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are SLSes, SA, TS, and EAs.
- moptipy** is the *Metaheuristic Optimization in Python* library⁸⁵. It has been used in several different research works, including^{13,49–51,73,90,94,95}. Algorithms implemented by moptipy include EAs, $(1+1)$ EA, RLS, SA, and the $(1+1)$ FEA, amongst others. Learn more at <https://thomasweise.github.io/moptipy> and <https://thomasweise.github.io/moptipyapps>.

Glossary IV



- OneMax** OneMax is the simplest possible benchmark problem for bit-string based black-box search^{23,56,93}. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on OneMax is in $\Theta(n \ln n)$ ^{21,56}. The average runtime $(1+1)$ FEA on OneMax seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^2 \ln n)$ ⁸⁶.
- Plateau** The Plateau problem is a benchmark problem for bit-string based black-box search^{1,93}. It is structured exactly like Jump, but instead of being deceptive, the last ω bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and ω otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the neutral plane around the optimum that an algorithm needs to cross. An $(1+1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus¹. The average runtime of the $(1+1)$ FEA on Plateaus is slower by a factor seemingly proportional in n .
- PostgreSQL** An open source object-relational DBMS^{25,58,62,72}. See <https://postgresql.org> for more information.
- Python** The Python programming language^{36,48,54,79}, i.e., what you will learn about in our book⁷⁹. Learn more at <https://python.org>.
- QAP** The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations^{9,13,43,52}. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP, a distance matrix D is given, where D_{pq} specifies the distance from location p to location q , as well as a flow matrix F , where F_{ij} is the amount of material flowing from facility i to facility j . The objective function $f(x)$ then rates a permutation x as $f(x) = \sum_{i=1}^n \sum_{j=1}^n D_{x_i x_j} F_{ij}$. The QAP is $\mathcal{NP}$ -complete⁷⁰.

Glossary V



- RLS Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.
- FRLS The RLS with FFA plugged in ⁹³. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1 + 1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1 + 1)$ FEA.
- SA Simulated Annealing is a SLS that sometimes accepts a worse solution ^{11,39,40,60}. The probability to do so decreases over time and with the difference in objective values, i.e., is the lower the worse the new solution is.
- SLS *Stochastic local search or local search* for short, is one of the simplest metaheuristic search paradigms ³⁵: A single “current” candidate solution is maintained and new solutions are sampled from its neighborhood. If these solutions are deemed interesting based on their objective value, they replace the “current”. The cycle continues. Basic examples are RLS and $(1 + 1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.
- Trap The Trap problem is a benchmark problem for bit-string based black-box search ^{22,57,93}. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the “gradient” in the search space leads the search away from the global optimum. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = 0$ and $n - \|x\|_1 - 1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ²². In experiments, the $(1 + 1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{86,87,93}.
- TS *Tabu Search* is one of the most successful SLSes ^{29,30,32}. In order to avoid returning to previous solutions, it declares certain solutions or search moves as “tabu.” For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion).

Glossary VI



TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are defined^{2,34,47,49,80,88}. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$ -hard combinatorial optimization problems^{28,34}.

TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP)^{66,67}. Its original URL was available until 2025 and then became defunct. The repository <https://github.com/shredderzvj/TSPLIB> contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite>, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of symmetric TSPs with scales n ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric TSPs with scales $n \in 17..443$.

TTP The *Traveling Tournament Problem* is the combinatorial optimization problem of both efficiently and fairly organizing a tournament of n teams that play against each other in a pairwise fashion^{24,90}. The efficient part boils down to arranging the games such that the total travel length is short, which is somewhat similar to the classical TSP. Initially, each team is at its home location. On each day, a team needs to travel if its scheduled game is not at its present location. On the last day, each team may need to travel back home unless their last game is a home game. The total travel length sums up the lengths of all travels over all teams. The fair part is represented in several constraints, such as doubleRoundRobin, compactness, maxStreak, and noRepeat. The TTP is $\mathcal{NP}$ -hard⁷⁵.

TwoMax The TwoMax problem is a benchmark problem for bit-string based black-box search^{27,74,93}. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\{\|x\|_1, n - \|x\|_1\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$ ^{26,27}. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax^{86,87,93}.

URI A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet⁹². It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples for URIs.

Glossary VII



URL A *Uniform Resource Locator* identifies a resource in the World Wide Web (WWW) and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites ⁶. URLs are subsets of Uniform Resource Identifiers (URIs).

WWW World Wide Web ^{5,19}

$\circ^*$ the superscript $\star$ symbol denotes that $\circ$ is a global optimum, i.e., there is no better solution than $\circ$.

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j . For example, $5..9$ is equivalent to $\{5, 6, 7, 8, 9\}$

$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution y in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

n We usually use n to denote the scale, i.e., the size of the problem to solve. For example, in a TSP, this may be the number of cities, in a discrete benchmark problem such as OneMax or Trap, this may be the number of bits over which the problem is defined, in a QAP, this may be the number of facilities and locations, and in a MaxSAT problem, this may be the number of variables that are input to the Boolean formula that we want to become true. There are also problems where the scale is not defined straightforwardly like this. For example, in a JSSP, we have a number of machines and a number of jobs, which together define the size of the problem. But even then, the scale n relates to the size of the solution space $\mathbb{Y}$, and, hence, also of the search space $\mathbb{X}$, which usually grow exponentially with n . And this is already the main issue: The larger n , the (exponentially) larger the number of possible solutions and thus, the larger the runtime of algorithms..

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$.

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer) ³³.

Glossary VIII



- $\mathcal{NP}$ -complete A decision problem is $\mathcal{NP}$ -complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time ^{33,64}. A problem is $\mathcal{NP}$ -complete if it is $\mathcal{NP}$ -hard and if it is in $\mathcal{NP}$.
- $\mathcal{NP}$ -hard Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$ -hard problems ^{12,15,46} have a worst-case runtime that grows exponentially with the problem scale. A problem is $\mathcal{NP}$ -hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time ³³.
- $\Omega(g(x))$ If $f(x) = \Omega(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0 \forall x \geq x_0$ ^{41,42}. In other words, $\Omega(g(x))$ describes a lower bound for function growth.
- $\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x) \forall x \geq x_0$ ^{3,41,42,45}. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.
- $\Theta(g(x))$ If $f(x) = \Theta(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \Omega(g(x))$ ^{41,42}. In other words, $\Theta(g(x))$ describes an exact order of function growth.
- $\mathbb{R}$ the set of the real numbers.
- $\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.
- $\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions y . These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.
- y a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.
- $\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$, and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.